

Advanced Data Analytics — Lecture 13

Dimitrios Karyampas

Department of Economics, University of Lausanne, Switzerland

Slides by Simon Scheidegger (2023–2025), adapted with permission

December 7th, 2026 | 12:30 - 14:00: Internef 125 | 16:30 - 18:00: Anthropole 3185

Roadmap: today

- ▶ More advanced topics:
 - ▶ Reinforcement learning
 - ▶ Example: Markowitz Portfolio (self-study)
 - ▶ Example: Dynamic Programming with Gaussian Processes (discussed)
- ▶ Throughout lectures - hands-on:
 - ▶ Examples related to the day's topics in Tensorflow and scikit-learn
 - ▶ Final thoughts

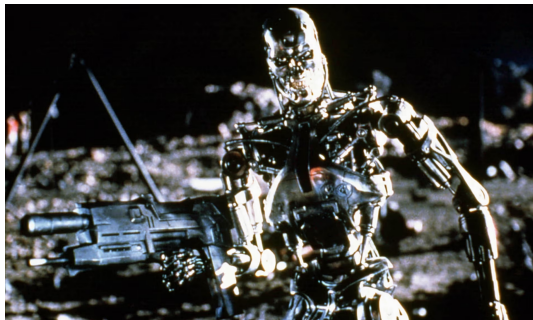
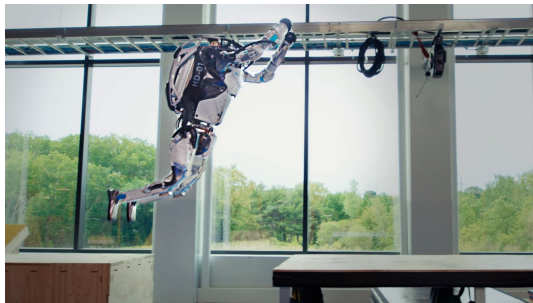
Self-Driving Cars

- ▶ Carnegie Mellon University — 1990's:
Self Driving Cars S1E2: ALVINN.
- ▶ Google — 2017: Waymo.
- ▶ Classification.
- ▶ Regression.
- ▶ Reinforcement learning.
- ▶ Prediction.



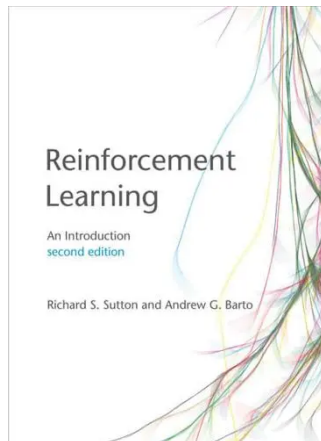
Two-Legged Robots

Boston Dynamics' Atlas Robot Can Do Parkour.



Reinforcement Learning

- ▶ <https://www.alphagomovie.com>
- ▶ https://youtu.be/8tq1C8spV_g
- ▶ Nice and complete tutorial: <https://umw.davidsilver.uk/teaching/>



Recall — Classes of Learning

- ▶ Supervised Learning
- ▶ Data: (x, y) , where x is data, y is label
- ▶ Goal: Learn function to map $x \rightarrow y$
- ▶ Example (Classification):



This thing is a banana

Recall — Classes of Learning

- ▶ Unsupervised Learning
- ▶ Data: x — x is data, no labels
- ▶ Goal: Learn underlying structure
- ▶ Example:



This thing is like the other thing

Reinforcement Learning

- ▶ Reinforcement Learning (RL)
- ▶ Data: state-action pairs
- ▶ Goal: Act optimally in a given environment. Maximize future rewards over many time steps
- ▶ Example:



Eat this thing because it will keep you alive

RL Key Concepts

See Marsland (2014), Sutton & Barto (2018)



Agent

Agent — takes actions

RL Key Concepts

See Marsland (2014), Sutton & Barto (2018)



Agent



Environment

Environment: the world in which the agent exists and operates

RL Key Concepts

See Marsland (2014), Sutton & Barto (2018)



Agent

Actions



Action: a_t



Environment

Action: a (possible) move the agent can make in the environment.
Actions for now: discrete (limited amount of possible actions for now).

RL Key Concepts

See Marsland (2014), Sutton & Barto (2018)



Agent

← Observations

Actions →

Action: a_t



Environment

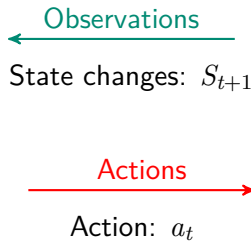
Observations: of the environment after taking actions.

RL Key Concepts

See Marsland (2014), Sutton & Barto (2018)



Agent



Environment

State: a situation which the agent perceives.

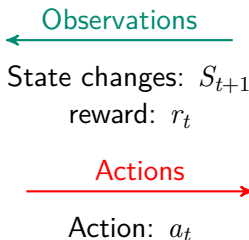
“A concrete situation where the agent is in”, measured, e.g., by a sensor.

RL Key Concepts

See Marsland (2014), Sutton & Barto (2018)



Agent



Environment

Reward: feedback that measures the success or failure of the agent's action in the environment.

- ▶ **“Good action”** : high reward.
- ▶ **“Poor action”** : low reward.

Rewards: can be immediate, or they can be delayed!

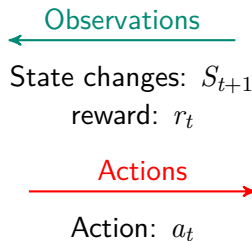
Example: Getting a reward



RL Key Concepts



Agent



Environment

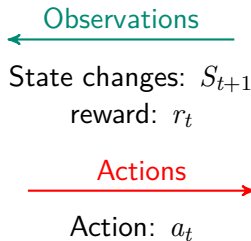
$$R_t = \sum_{i=t}^{\infty} r_i$$

Total reward

RL Key Concepts



Agent



Environment

$$R_t = \sum_{i=t}^{\infty} r_i = r_t + r_{t+1} \dots + r_{t+n} + \dots$$

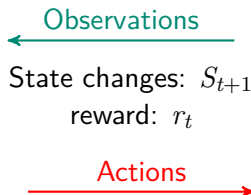
Total reward

Problem: Want to avoid infinitely large rewards!

RL Key Concepts



Agent



Action: a_t



Environment

$$R_t = \sum_{i=t}^{\infty} \gamma^i r_i$$

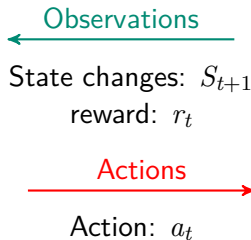
Discounted total reward

Problem: Want to avoid infinitely large rewards → use discount factor!

RL Key Concepts



Agent



Environment

$$R_t = \sum_{i=t}^{\infty} \gamma^i r_i = \gamma^t r_t + \gamma^{t+1} r_{t+1} \dots + \gamma^{t+n} r_{t+n} + \dots$$

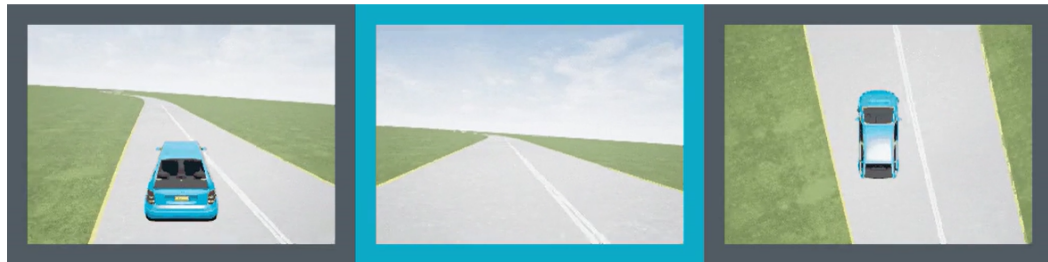
γ : discount factor

Discounted total reward

Problem: Want to avoid infinitely large rewards → use discount factor!

Example: Learn to drive

<https://wayve.ai/blog/learning-to-drive-in-a-day-with-reinforcement-learning>



Self-driving car

- ▶ **Action:** Steering angle
- ▶ **Next State:** Next camera input that the car sees.

The Q-function

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Total reward, R_t is the discounted sum of all rewards obtained from time t .

$$Q(s, a) = \mathbb{E}[R_t]$$

The Q-function captures the expected total future reward an agent in state, s , can receive by executing a certain action, a .

How to take actions given a Q-function?

$$\underbrace{Q(s, a)}_{(state, action)} = \mathbb{E}[R_t]$$

Ultimately, the agent needs a policy $\pi(s)$, to infer the best action to take at its state s .

Strategy: the policy should choose an action that maximizes the future reward:

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

Two types of RL Algorithms

Value Learning:

Find $Q(s, a)$

$a = \operatorname{argmax}_a Q(s, a)$

Policy Learning:

Find $\pi(s)$

Sample $a \sim \pi(s)$

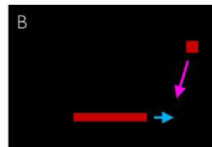
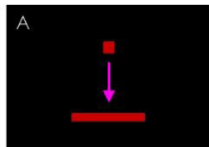
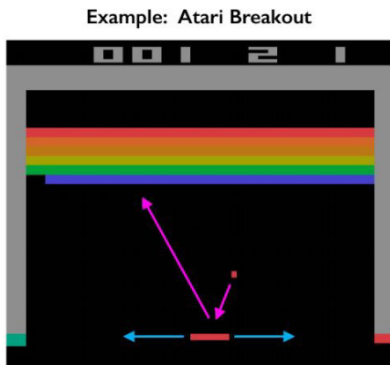
Value Learning:

Find $Q(s, a)$

$$a = \operatorname{argmax}_a Q(s, a)$$

Looking closer into the Q-function

<https://keras.io/examples/rl/>

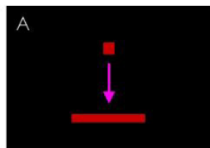
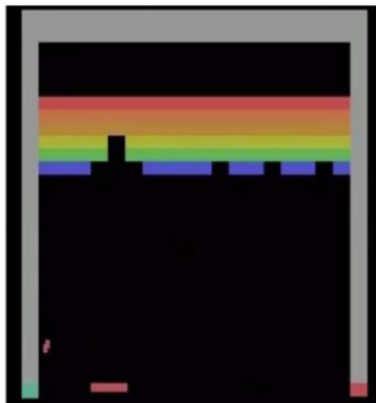


- ▶ It can be very difficult for humans to Accurately estimate Q-values.
- ▶ Which (s, a) pair has a higher Q-value?

Looking closer into the Q-function

<https://keras.io/examples/rl/>

Example: Atari Breakout - Middle

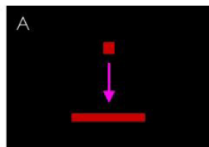
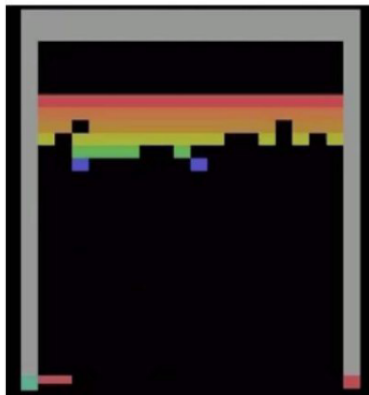


- ▶ It can be very difficult for humans to Accurately estimate Q-values.
- ▶ Which (s, a) pair has a higher Q-value?

Looking closer into the Q-function

<https://keras.io/examples/r1/>

Example: Atari Breakout - Side



- ▶ It can be very difficult for humans to Accurately estimate Q-values.
- ▶ Which (s, a) pair has a higher Q-value?

Atari Breakout



<https://www.youtube.com/watch?v=V1eYniJ0Rnk>

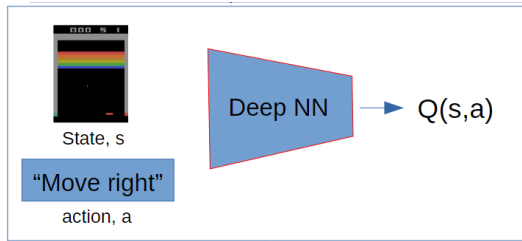
Misspecified Reward Function



- ▶ <https://www.youtube.com/watch?v=t10IHko8ySg>
- ▶ <https://openai.com/research/faulty-reward-functions>

Example: RL and DNN

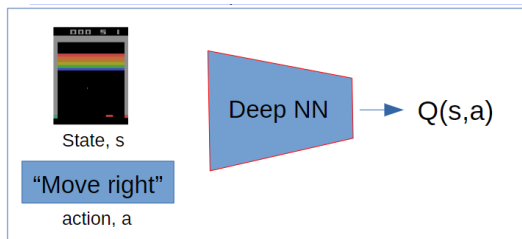
How can we use deep neural networks to model Q-functions?



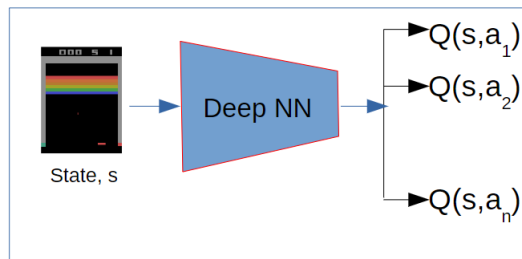
Problem: for a given state s , we need to compute Q for all possible $a \rightarrow$ computationally "expensive"

Example: RL and DNN

How can we use deep neural networks to model Q-functions?



Action & State $\rightarrow$ expected return
Problem: for a given state s , we need to compute Q for all possible $a \rightarrow$ computationally "expensive"

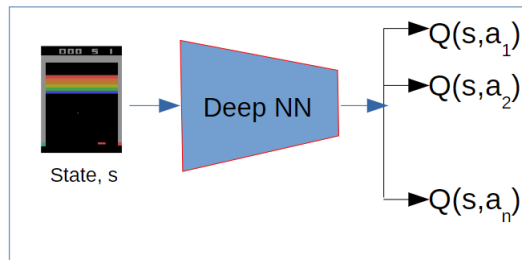
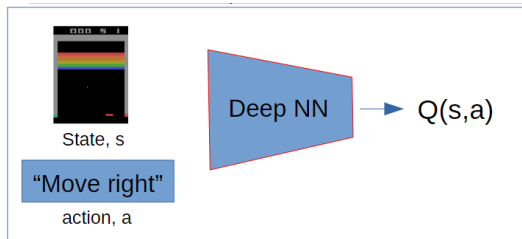


State $\rightarrow$ Expected return for each action.
Alternative approach.

- $\rightarrow$ Multiple outputs.
- $\rightarrow$ Take max.

Example: RL and DNN

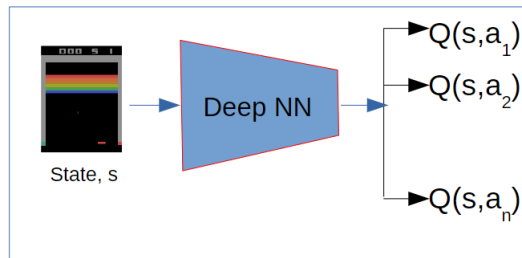
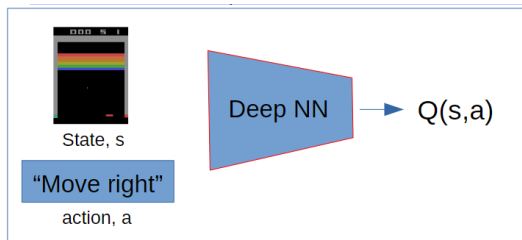
How can we use deep neural networks to model Q-functions?



$$\mathcal{L} = \mathbb{E} \left[\left\| \left(r + \gamma \max_{a'} Q(s', a') \right) - Q(s, a) \right\|^2 \right]$$

Example: RL and DNN

How can we use deep neural networks to model Q-functions?

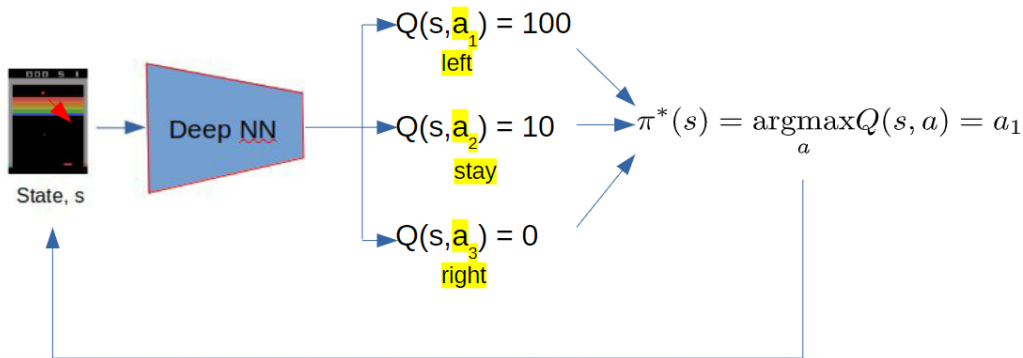


Value you observed when taking a particular action

$$\mathcal{L} = \mathbb{E} \left[\left\| \overbrace{\left(r + \gamma \max_{a'} Q(s', a') \right)}^{\text{target}} - \overbrace{Q(s, a)}^{\text{predicted}} \right\|^2 \right]$$

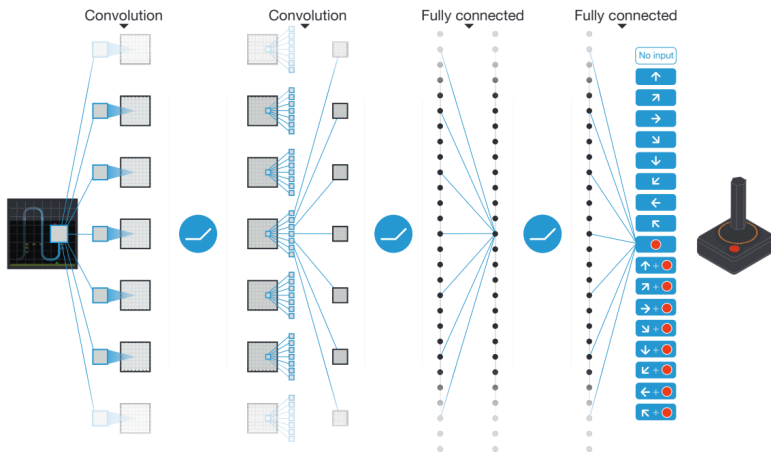
Output of NN

Example: RL and DNN



Train a DQN for the Atari test case

<http://deepmind.com/>

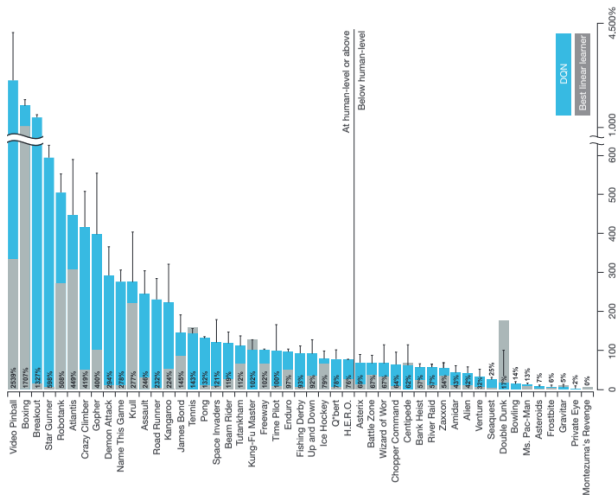


On the left — state of the game, on the right — possible actions

Atari: Super Human Performance

Mnih et al. (2015) —

<https://web.stanford.edu/class/psych209/Readings/MnihEtAlHassibis15NatureControlDeepRL.pdf>



Some problems with DQN

- ▶ Trouble with Complexity:
 - ▶ Can model scenarios where the action space is discrete and small
 - ▶ Cannot handle continuous action spaces — *IMPORTANT: Imagine you want to predict the steering wheel angle of a car!*
- ▶ Trouble with Flexibility:
 - ▶ Cannot learn stochastic policies since policy is deterministic computed from the Q-function (take argmax).
 - ▶ To overcome, consider a new class of RL training algorithms: Policy gradient methods.

Recall: Policy Learning

Policy Learning:

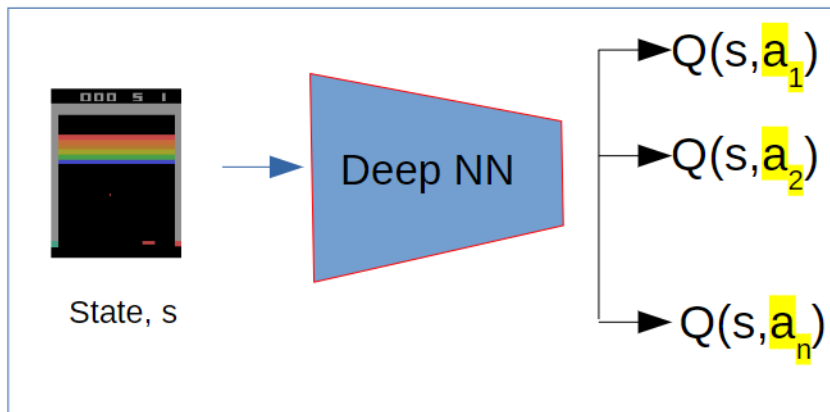
Find $\pi(s)$

Sample $a \sim \pi(s)$

The basic idea of Policy Gradient

<https://www.davidsilver.uk/wp-content/uploads/2020/03/pg.pdf>

DQN (before): Approximating Q and inferring the optimal policy

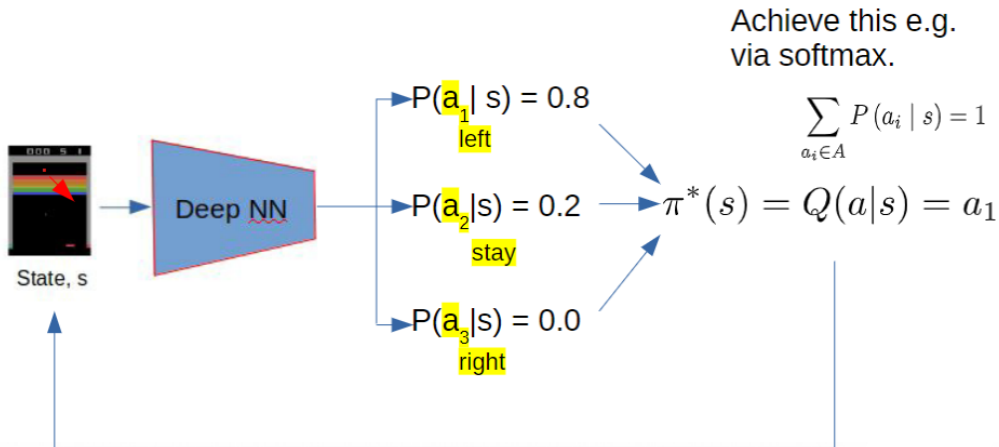


The basic idea of Policy Gradient

<https://www.davidsilver.uk/wp-content/uploads/2020/03/pg.pdf>

DQN (before): Approximating Q and inferring the optimal policy

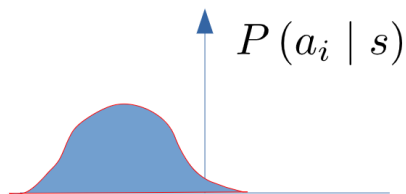
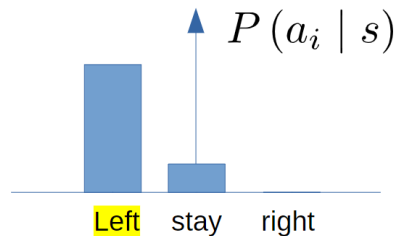
Policy Gradient: Directly optimize the policy!



Discrete and Continuous Actions

<https://www.davidsilver.uk/wp-content/uploads/2020/03/pg.pdf>

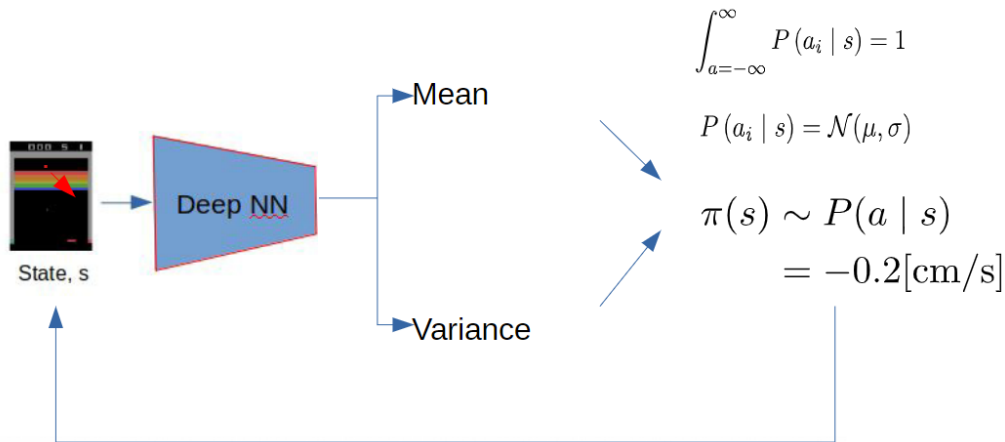
- ▶ Discrete action space
 - ▶ Where should I move? (left, right, stay)
- ▶ Continuous action space
 - ▶ How fast should I move (velocity vector)



← speed to the left speed to the right →

Example: RL and DNN

<https://www.davidsilver.uk/wp-content/uploads/2020/03/pg.pdf>



Policy Gradient: Directly model the continuous action space.

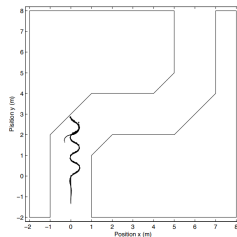
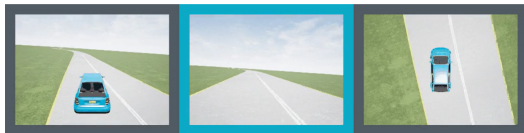
Training Policy Gradients

<https://wayve.ai/thinking/learning-to-drive-in-a-day/>,

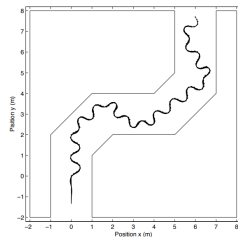
<https://www.davidsilver.uk/wp-content/uploads/2020/03/pg.pdf>

Algorithm:

- ▶ Initialize the agent (car)
- ▶ Run a policy until termination.
- ▶ Record all states, actions, and rewards.
- ▶ Decrease the probability of actions that resulted on a low reward.
- ▶ Increase probability of actions that resulted in a high reward.



(a) Before learning



(b) After learning

Training Policy Gradients

<https://wayve.ai/thinking/learning-to-drive-in-a-day/>,

<https://www.davidsilver.uk/wp-content/uploads/2020/03/pg.pdf>

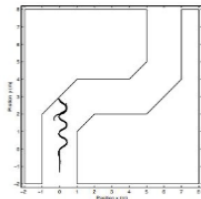
Algorithm:

- ▶ Initialize the agent (car)
- ▶ Run a policy until termination.
- ▶ Record all states, actions, and rewards.
- ▶ Decrease the probability of actions that resulted on a low reward.
- ▶ Increase probability of actions that resulted in a high reward.

$$\text{loss} = - \underbrace{\log P(a_t | s_t)}_{\text{Log-likelihood of action}} \underbrace{R_t}_{\text{Reward}}$$

$$w' = w - \nabla \text{loss}$$

$$w' = w + \underbrace{\nabla \log P(a_t | s_t) R_t}_{\text{Policy gradient loss}}$$



Connection to Dynamic Programming

The solution is approached in the limit as $j \rightarrow \infty$ by iterations on at every coordinate of the "grid" of observations.

$$V_{j+1}(x) = \max \{ r(x, u) + \beta V_j(\tilde{x}) \}$$
$$\tilde{x} = g(x, u)$$

where:

- ▶ x : grid point/observation/state variable. Describes your problem.
- ▶ $V_j(\tilde{x})$: 'old solution' on which we Interpolate (i.e., "predict").

RL with Available Tools

- ▶ Keras & TF resources
 - ▶ <https://keras.io/examples/rl>
- ▶ Deep Policy Gradient
 - ▶ https://keras.io/examples/rl/ddpg_pendulum/
 - ▶ If time permits, let's look at the code: `demo/06_ddpg_pendulum.ipynb`
 - ▶ <https://arxiv.org/pdf/1509.02971.pdf>
- ▶ Atari Breakout
 - ▶ https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/rl/ipynb/deep_q_network_breakout.ipynb

Action Required: Portfolio Optimization

- ▶ `demo/Least_square_interpolation_Markowitz.ipynb`
- ▶ Recall the problem of an investor who starts with an initial wealth $W_0 = 1$ at time $t = 0$ and, at each period $t = 0, \dots, T - 1$ allocates a fraction $u_t = u_t(S_t)$ of the total portfolio value to the risky asset, and the remaining fraction $1 - u_t$ is invested in a risk-free bank account that pays a risk-free interest rate $r_f = 0$.
- ▶ If the wealth process is self-financing and the one-step rewards R_t for $t = 0, \dots, T - 1$ are the risk-adjusted portfolio returns.

$$R_t = r_t - \lambda \text{Var} [r_t \mid S_t] = u_t \phi_t - \lambda u_t^2 \text{Var} [\phi_t \mid S_t]$$

Portfolio Optimization

- ▶ then the optimal investment problem for $T - 1$ steps is given by

$$V^\pi(s) = \max_{u_t} \mathbb{E} \left[\sum_{t=0}^T R_t \mid S_t = s \right] = \max_{u_t} \mathbb{E} \left[\sum_{t=0}^T u_t \phi_t - \lambda u_t^2 \text{Var} [\phi_t \mid S_t] \mid S_t = s \right]$$

- ▶ where we allow for short selling in the ETF (i.e., $u_t < 0$) and borrowing of cash $u_t > 1$.
- ▶ We apply the LSPI algorithm to $N = 2000$ simulated stock prices over $T = 10$ periods:

$$\left\{ \left\{ S_t^{(i)} \right\}_{i=1}^N \right\}_{t=1}^T$$

Portfolio Optimization

- At each time period, we construct a basis

$$\{\Psi_k(s, a)\}_{k=1}^K$$

over the state-action space using $K = 256$ B-spline basis functions, where

$$s \in \left[\min \left(\left\{ S_t^{(i)} \right\}_{i=1}^N \right), \max \left(\left\{ S_t^{(i)} \right\}_{i=1}^N \right) \right] \text{ and } a \in [-1, 1]$$

- Note that in this particular simple problem, the actions are independent of the state space and hence the basis construction over state-action space is not really needed.
- However, our motive here is to show that we can obtain an estimate close to the exact solution

$$w_t^* = \frac{\mathbb{E}[\phi_t]}{2\lambda \text{Var}[\phi_t]}$$

using a more general methodology.

Portfolio Optimization

- ▶ a_{T-1} is initialized with uniform random samples at time step $t = T - 1$.
- ▶ In subsequent time steps, at , $t \in \{T - 1, T - 2, \dots, 0\}$, we initialize with the previous optimal action, $a_t = a^*_{t+1}$
- ▶ LSPI updates the policy iteratively $\pi^{k-1} \rightarrow \pi^k$ until convergence of the value function.
- ▶ The Q-function is maximized over a gridded state-action space, Ω^h , with 200 stock values and 20 action values, to give the optimal action

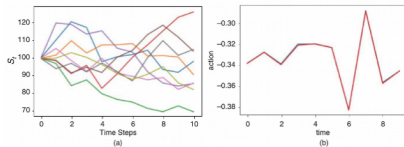
$$a_t^k(s) = \pi_t^k(s) = \underset{a}{\operatorname{argmax}} Q_t^{\pi^{k-1}}(s, a), (s, a) \in \Omega^h$$

Portfolio Optimization

- For each time step, LSPI updates the policy π^k until the following stopping criterion is satisfied

$$\left\| V^{\pi^k} - V^{\pi^{k-1}} \right\|_2 \leq \tau \text{ where } \tau = 1 \times 10^{-6}$$

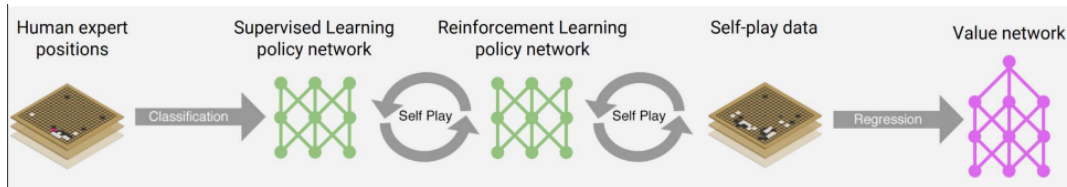
- The optimal allocation using the LSPI algorithm (red) is compared against the exact solution (blue).



- Stock prices are simulated using an Euler scheme over a one year horizon. At each of ten periods, the optimal allocation is estimated using the LSPI algorithm (red) and compared against the exact solution (blue). (a) Stock price simulation. (b) Optimal allocation.

Alpha Go Beats Human Player

Silver et al. (2016) — <https://www.nature.com/articles/nature16961>



Alpha Go Beats Human Player

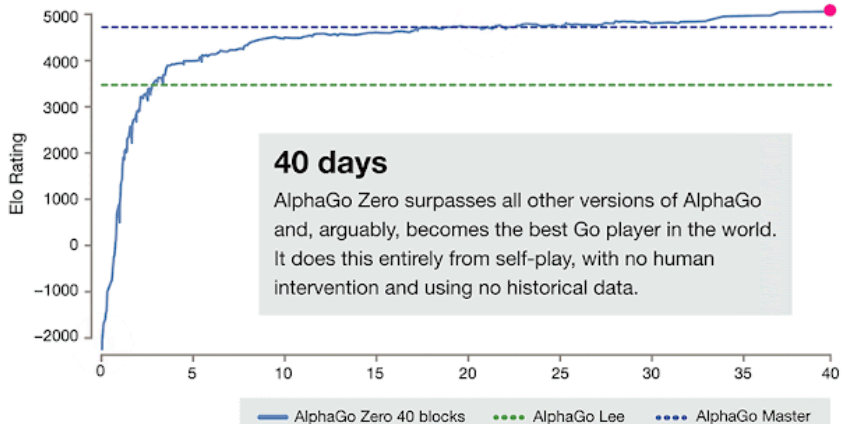
Silver et al. (2016) — <https://www.nature.com/articles/nature16961>



AlphaZero: RL from Self-Play (2018)

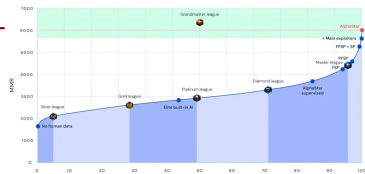
<https://deepmind.google/discover/blog/alphago-zero-starting-from-scratch/> and

<https://www.science.org/doi/10.1126/science.aar6404>



Grandmaster Level in Starcraft II using Multi-Agent RL

- <https://www.nature.com/articles/s41586-019-1724-z>
- <https://deepmind.com/blog/article/alphastar-mastering-real-time-strategy->



Questions?

